

Otimização de Telemetria com Buffer Circular

Comparação entre alocação dinâmica $O(n)$ e buffer circular $O(1)$
em uma arquitetura cliente-servidor com MQTT

Silvio Fittipaldi

Bernardo Heuer

Rodrigo Nunes

8 de junho de 2026

Resumo

Este trabalho compara duas estratégias de gerência de memória para um fluxo de telemetria de alta frequência: uma estrutura de *alocação dinâmica* (anti-padrão, custo $O(n)$ por inserção) e um *buffer circular* com índices `head/tail` e operações $O(1)$. O enunciado original é embarcado (ESP32); aqui ele é reproduzido em uma arquitetura cliente-servidor de software, preservando todos os objetivos didáticos: duas vertentes comparadas, transmissão MQTT, modelo produtor-consumidor, instrumentação de tempo e memória, testes de escala ($N = 100/5.000/20.000/100.000$), painel com gráficos de dados e de performance, e a presente análise. Os resultados confirmam empiricamente o contraste assintótico esperado e evidenciam o impacto da instabilidade de rede.

1 Introdução e adaptação

O problema clássico de telemetria embarcada expõe uma disparidade temporal: a captura de sensores ocorre na escala de microssegundos, enquanto a transmissão em rede ocorre na escala de milissegundos. Se o produtor de dados ficar acoplado ao consumidor de rede, a amostragem é estrangulada pela latência da rede; pior, se a estrutura que acumula amostras crescer ou se reorganizar a cada inserção, o custo de CPU e de memória degrada de forma super-linear.

Como o ambiente de execução não dispõe de hardware embarcado, o ESP32 foi substituído por um **cliente em C++17** que gera amostras simuladas em alta frequência. A equivalência entre o enunciado e esta versão está na Tabela 1.

Tabela 1: Equivalência entre o enunciado embarcado e a versão cliente-servidor.

Conceito (embarcado)	Equivalente (cliente-servidor)
ESP32 capturando sensor	Cliente produtor em C++ (amostras simuladas)
Captura (μs) $\times$ rede (ms)	Geração em memória (μs) $\times$ publicação MQTT (ms)
Transmissão MQTT	Mantida: publicação em broker Mosquitto
<code>micros()</code>	<code>QueryPerformanceCounter</code> / <code>steady_clock</code> ($\mu s/ns$)
<code>ESP.getFreeHeap()</code> / fragmentação	Contadores globais de <code>new/delete</code> + RSS do SO
Colapso por <i>stack overflow</i>	Crescimento ilimitado de memória / risco de OOM
Serial Plotter	Dashboard web (Node-RED)

O servidor é composto pelo broker Eclipse Mosquitto [4] e pelo dashboard Node-RED [6], ambos em contêineres Docker, comunicando-se pelo protocolo MQTT [5].

Adaptações de ambiente. Duas escolhas de engenharia, no mesmo espírito da adaptação acima, tornaram o projeto autossuficiente: (i) a camada MQTT foi implementada como um cliente MQTT 3.1.1 [5] mínimo sobre *sockets* TCP (Winsock/BSD), preservando o contrato `publishSync` (bloqueante) versus `publishAsync` (não bloqueante) sem depender de bibliotecas

externas como o Eclipse Paho [3]; e (ii) o *build* usa `g++` diretamente, além de um `CMakeLists.txt` canônico. Nenhuma dessas escolhas altera os objetivos didáticos.

2 As duas vertentes

Vertente 1 — `DynamicBuffer` (anti-padrão, $O(n)$). Reproduz explicitamente o problema. Em duas sub-variantes:

- **SHIFT:** mantém uma janela de tamanho W ; ao encher, remove o primeiro elemento deslocando todos os demais (`erase(begin)`), custo $\Theta(W)$ por inserção.
- **REALLOC:** *realloc* manual a cada amostra — aloca um bloco com uma posição a mais, copia todo o conteúdo e libera o anterior. Custo $O(n)$ por inserção, crescimento ilimitado e N alocações.

Vertente 2 — `RingBuffer` (eficiente, $O(1)$). Classe própria com `head/tail` e `count`; aloca o armazenamento **uma única vez** no construtor e nunca realoca. As operações usam apenas aritmética modular (`idx = (idx + 1) mod cap`): não há `erase`, `insert(begin)`, `memmove` nem `realloc`. Política de *overflow*: sobrescrever o mais antigo (telemetria prioriza dados recentes).

3 Análise Assintótica

A análise abaixo emprega a notação assintótica padrão [1].

Vertente 1 (SHIFT). A inserção com manutenção de janela executa um `push_back` amortizado $O(1)$ seguido, quando a janela está cheia, de um deslocamento de $W - 1$ elementos. Logo o custo por inserção é $\Theta(W)$ e o custo total de N inserções é

$$T_{\text{shift}}(N) = \Theta(N \cdot W).$$

Vertente 1 (REALLOC). A i -ésima inserção copia os $i - 1$ elementos já existentes mais um, ou seja, $\Theta(i)$. O custo total é

$$T_{\text{realloc}}(N) = \sum_{i=1}^N \Theta(i) = \Theta\left(\frac{N(N+1)}{2}\right) = \Theta(N^2),$$

e o número de alocações é exatamente N , com pico de bytes vivos proporcional a $N \cdot \text{sizeof}(\text{Sample})$.

Vertente 2 (`RingBuffer`). Cada `push` e cada `pop` fazem trabalho constante (escrita/leitura mais incremento modular):

$$T_{\text{ring}}(N) = \Theta(N), \quad \text{com 1 alocação total.}$$

O dreno em lote de k itens (`popBatch`) custa $\Theta(k)$, somando $\Theta(N)$ ao longo da execução.

A Figura 1 confirma o contraste: a latência média de inserção da Vertente 1 cresce com N/W , enquanto a da Vertente 2 permanece praticamente constante (no piso de resolução do relógio de alta precisão [2]). A Figura 2 mostra a distribuição completa por inserção, e a Figura 3 mostra a *jitter* (desvio-padrão da latência), também maior na vertente $O(n)$.

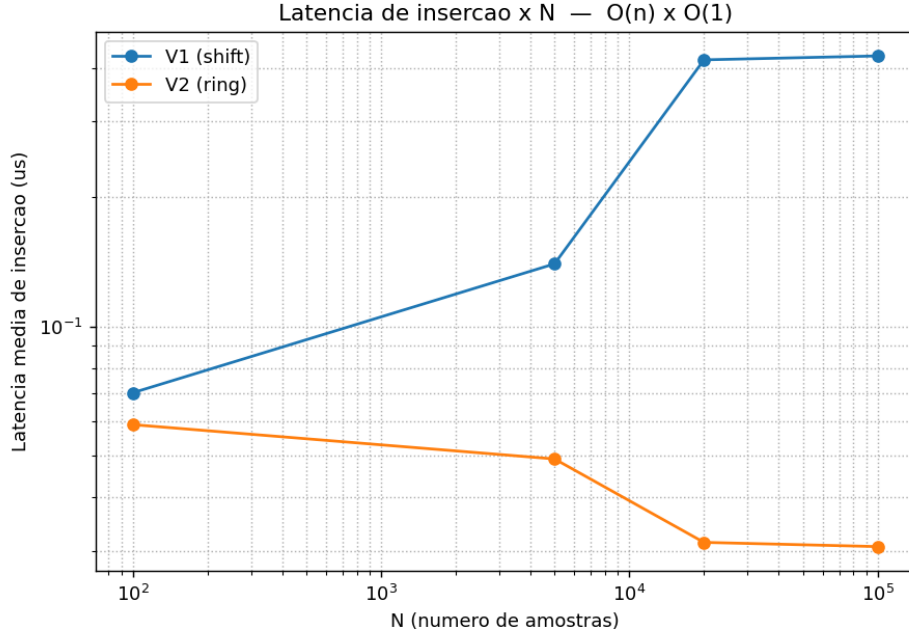


Figura 1: Latência média de inserção em função de N (eixos log-log). A Vertente 1 (**shift**, $O(n)$) cresce; a Vertente 2 (**ring**, $O(1)$) é plana.

4 Diagnóstico de Memória

A métrica primária de memória — portátil entre sistemas operacionais — são os **contadores globais de operador new/delete**, que interceptam toda alocação dinâmica do processo e atribuem à estrutura de dados apenas as alocações ocorridas durante suas inserções (isolando MQTT e *setup*). De forma complementar, registra-se o pico de RSS do processo.

A Tabela 2 resume os valores medidos. A Vertente 2 faz **uma** alocação para N até 100.000, com volume constante. A variante **realloc** da Vertente 1 faz N alocações e acumula bytes em ritmo quadrático — chegando a $\sim 4,8$ GB de bytes solicitados em $N = 20.000$ —, evidência direta da fragmentação e do crescimento que, no contexto embarcado, levariam ao colapso por falta de memória. As Figuras 4 e 5 ilustram o crescimento.

Tabela 2: Diagnóstico de memória (valores representativos medidos). **n_allocs** e **alloc_bytes** referem-se à estrutura de dados.

Configuração	N	n_allocs	alloc_bytes
V2 ring	100.000	1 ou 2	≈ 98 KB (constante)
V1 shift	100.000	12	≈ 98 KB
V1 realloc	5.000	5.000	≈ 300 MB
V1 realloc	20.000	20.000	$\approx 4,8$ GB

5 Discussão: instabilidade de rede e produtor–consumidor

O modelo de execução é a diferença central entre as vertentes. A Vertente 1 é **síncrona**: captura, insere ($O(n)$) e publica com **publishSync**, que bloqueia até o *round-trip* da rede; a amostragem *para* durante a latência. A Vertente 2 é **assíncrona**: o produtor captura e faz **push** $O(1)$ no **RingBuffer** (nunca bloqueia pela rede), enquanto um consumidor drena em lote e publica com **publishAsync**; a sincronização usa **mutex** e **condition_variable**.

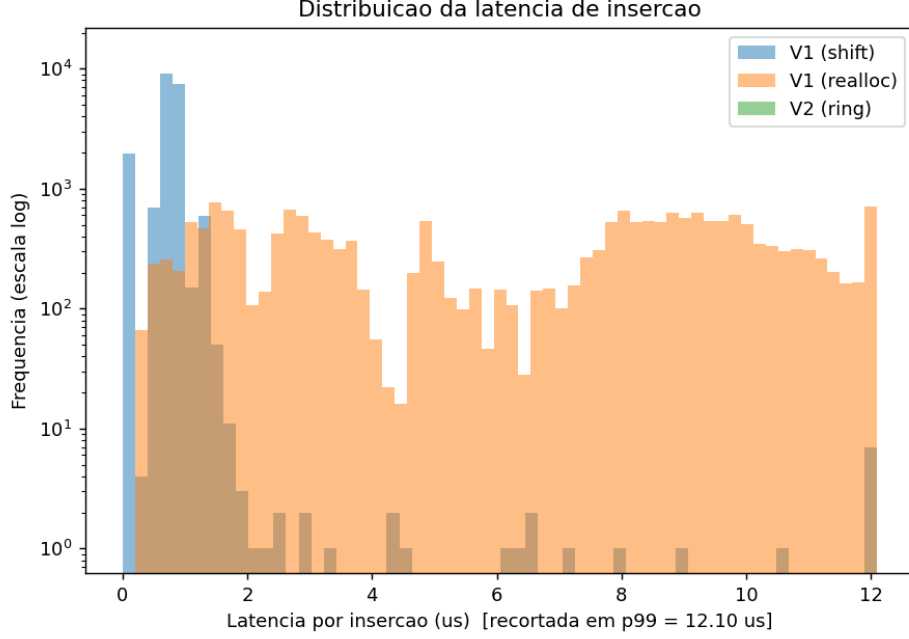


Figura 2: Distribuição da latência por inserção. A cauda longa da variante `realloc` reflete as cópias de arrays cada vez maiores.

Para simular instabilidade, injeta-se um atraso artificial por publicação (`-net-delay-ms`). A Figura 6 mostra o efeito sobre a *taxa de amostragem*: com apenas 1 ms de atraso, a Vertente 1 desaba de $\sim 9 \times 10^5$ para ~ 64 amostras/s (a captura fica refém da rede), enquanto a Vertente 2 mantém a amostragem em 10^5 – 10^6 amostras/s.

Trade-off da política de overflow. O desacoplamento tem um custo: quando o consumidor não acompanha (rede lenta), o `RingBuffer` satura e **sobrescreve as amostras mais antigas**. Ou seja, a Vertente 2 preserva a *taxa de captura* sacrificando dados antigos, ao passo que a Vertente 1 preserva *todas* as amostras sacrificando a taxa. Para telemetria, em que os dados recentes valem mais, a política de sobrescrita do buffer circular é a escolha adequada; o tamanho do buffer dimensiona quanta latência de rede o sistema absorve antes de começar a descartar.

6 Conclusão

Os experimentos confirmam, de ponta a ponta, a tese do trabalho. No tempo, a inserção passa de $O(n)$ (até $O(N^2)$ acumulado na variante `realloc`) para $O(1)$ com o buffer circular. Na memória, passa-se de N alocações com crescimento quadrático de bytes para uma *única* alocação de volume constante. Sob rede instável, o modelo produtor–consumidor com buffer circular mantém a taxa de amostragem ordens de grandeza acima da abordagem síncrona, ao preço de uma política de descarte de dados antigos bem definida. O buffer circular com aritmética modular é, portanto, a estrutura adequada para telemetria de alta frequência sujeita a latência de rede variável.

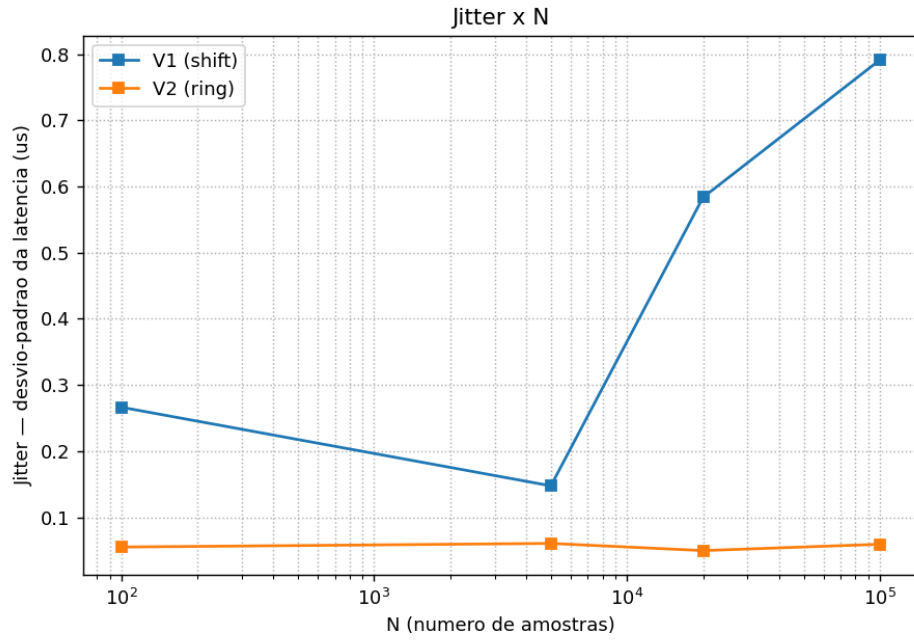


Figura 3: Jitter (desvio-padrão da latência de inserção) em função de N .

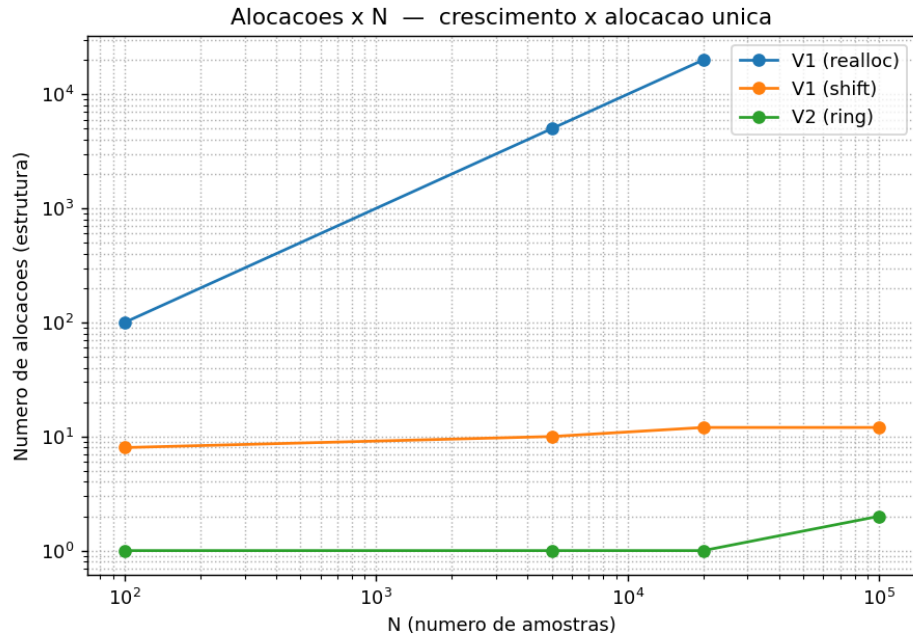


Figura 4: Número de alocações atribuídas à estrutura em função de N (log-log). A alocação única do `ring` contrasta com o crescimento linear do `realloc`.

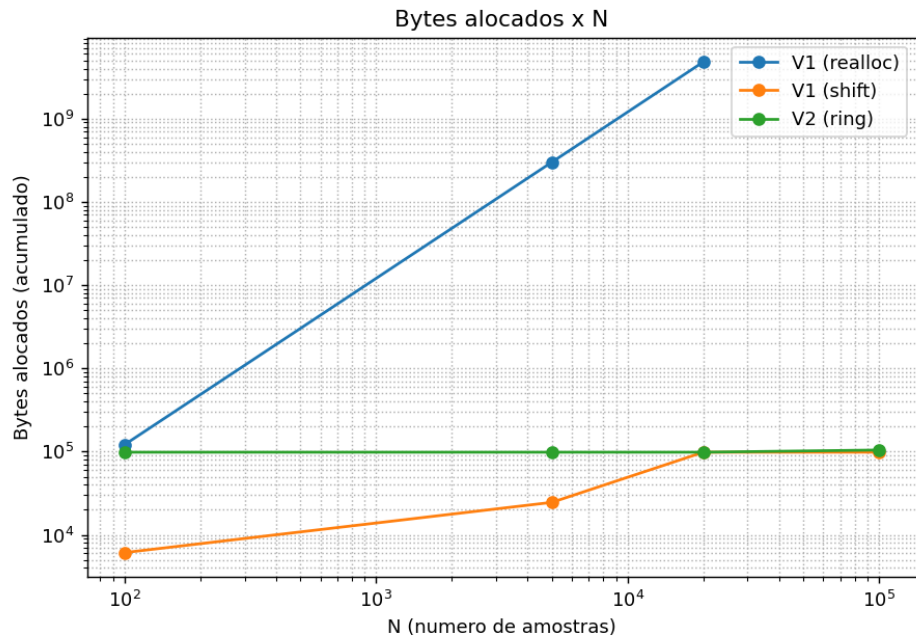


Figura 5: Bytes alocados (acumulado) em função de N (log-log).

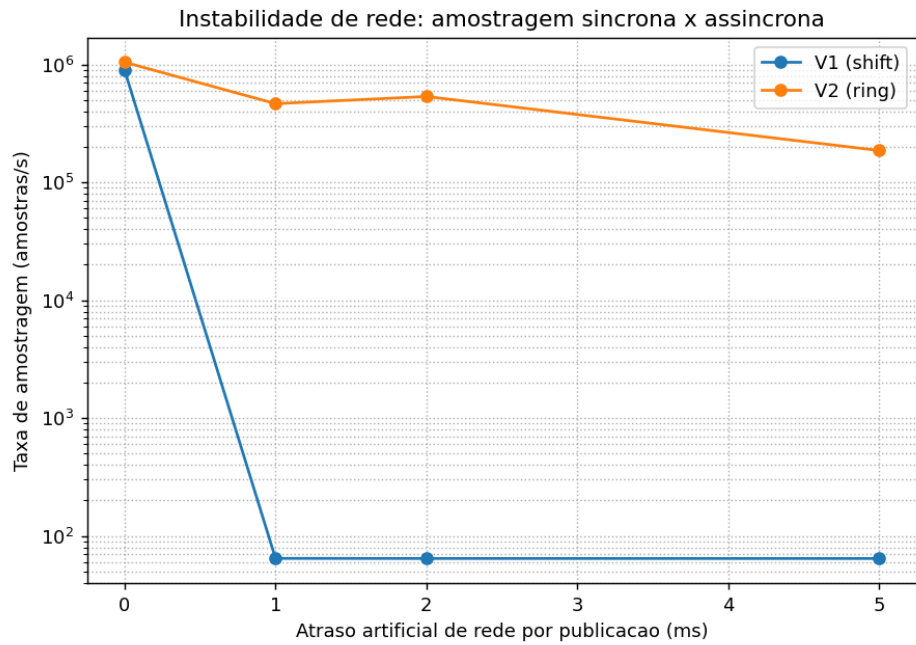


Figura 6: Taxa de amostragem sob atraso de rede. A vertente síncrona colapsa; a assíncrona absorve a latência no buffer.

Referências

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, 3 edition, 2009.
- [2] cppreference. `std::chrono::high_resolution_clock`. https://en.cppreference.com/w/cpp/chrono/high_resolution_clock, 2024.
- [3] Eclipse Foundation. Eclipse Paho MQTT C++ Client. <https://www.eclipse.org/paho/>, 2024.
- [4] Roger A. Light. Mosquitto: server and client implementation of the MQTT protocol. *Journal of Open Source Software*, 2(13), 265, 2017. <https://mosquitto.org>.
- [5] OASIS. MQTT Version 3.1.1. Technical report, OASIS Standard, 2014. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>.
- [6] OpenJS Foundation. Node-RED: Low-code programming for event-driven applications. <https://nodered.org>, 2024.